Distillate: A Meta-Agent Harness for Autonomous ML Research

Romain Lacombe
romain@distillate.dev

Abstract

Code generation agents are digital work execution machines, now running unsupervised for hours or days. Their immense promise to machine learning research rests on a measurable epistemic property: whether an agent can accurately forecast the outcomes of its own experiments before running them, update its beliefs from evidence, and compound those updates into scientific discovery. **In this paper, we demonstrate that modern CLI agents are well-calibrated autonomous ML researchers** when paired with a harness that addresses three coupled challenges: (1) coherence loss across long-horizon programs; (2) uncalibrated and irreproducible outcome predictions; and (3) thin human-in-the-loop oversight. We introduce **Distillate**, a multi-agent autonomous research steering and orchestration harness—and shipping Integrated Agents Environment (IAE)—addressing all three: Recursive Language Model (RLM) agents [Zhang et al., 2025] reasoning over typed Python namespaces in place of serialized prompts, pre-registration enforced as a tool contract with a cryptographic Git anchor, and a conversation-agent Orchestrator that translates researcher intent into a steerable experimental swarm. Empirically, across nine campaigns and n=326 pre-registered runs over three scientific domains (combinatorial mathematics, computer vision, and molecular biology), pre-registered predicted improvement $\Delta\hat{y}$ predicts realized improvement Δy at Pearson r=+0.35 (95% CI [+0.23, +0.49], p<0.001; r=+0.59 on GlycoGen, r=+0.40 on Tiny-MatMul, r=+0.28 on CIFAR-100 [Krizhevsky, 2009]), outperforming a frozen initial-belief baseline (r=+0.09, n.s.) by fourfold and a permutation null (r=+0.01). This forecasting signal replicates across three frontier model families (Claude Code, Codex CLI, and Gemini CLI), which we rank head-to-head via Sequential ELO, Bradley–Terry maximum likelihood, and a composite research score that weights peak performance, sample efficiency, and pre-registration accuracy. Self-rated confidence c, by contrast, is model-specific across frontier labs under both raw and sign-corrected metrics. Crucially, autonomous epistemic search translates into scientific discovery: a Distillate Experimentalist discovered a 50M-parameter discrete flow matching architecture that advanced the state of the art on de novo glycan structure prediction from 80.9% to 92.5% top-1 accuracy (+11.6 pp) under a leak-free protocol [Lacombe, 2026]. Our hope is that agent systems like Distillate will augment research rather than automate it, giving researchers command over larger scopes, harder questions, and more ambitious programs. If, tomorrow, researchers see further, perhaps it will be by standing on the shoulders of agents.

1 Introduction

Are modern CLI agents autonomous ML researchers? Code generation agents are digital work execution machines. Claude Code, Codex, and Gemini CLI now run unsupervised for hours or days, writing training scripts, evaluating results, and self-correcting as they progress, and a growing flurry

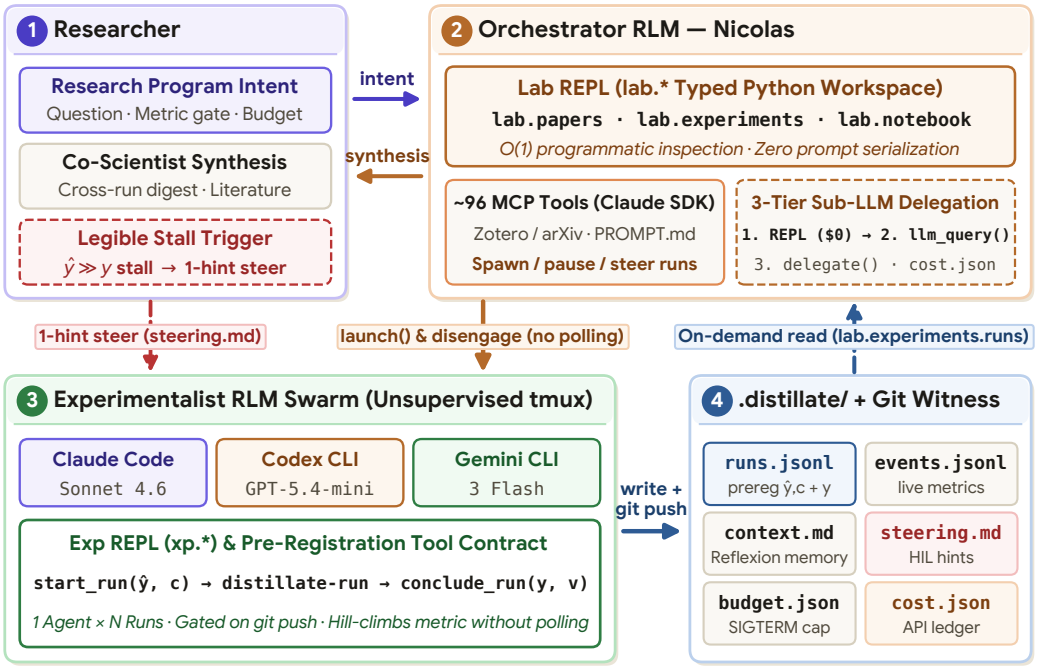


Figure 1: **Distillate meta-agent architecture.** (1) The researcher addresses high-level scientific *intent* and optional mid-flight steering hints to the Orchestrator RLM Agent (Nicolas). (2) The Orchestrator evaluates a typed Python facade in the Lab REPL without serializing the workspace into its prompt and spawns isolated sub-LLM calls via `llm_query()` and `delegate()`. (3) The Orchestrator launches an unsupervised Experimentalist RLM Swarm across interchangeable CLI backends (Claude Code, Codex CLI, Gemini CLI) and disengages. (4) Each Experimentalist hill-climbs in the auto-research loop (Figure 2) and commits pre-registrations and outcomes to `.distillate/` and Git, which the Orchestrator reads back on demand through `lab.experiments.runs`.

of agent-driven systems has produced novel algorithms [Romera-Paredes et al., 2024, Novikov et al., 2025, Trinh et al., 2024], chemistry campaigns [Boiko et al., 2023, Bran et al., 2024], biomolecule designs [Swanson et al., 2024], and multi-hour discovery cycles [Lu et al., 2024, Yamada et al., 2025, Edison Scientific, 2025]. Their immense promise to ML research rests on a measurable property: whether an agent can accurately forecast the outcomes of its own experiments before running them. We demonstrate that they can, and that three challenges decide whether autonomous research agents earn the compute they consume or waste it.

(i) Coherence. A research program spans hundreds of papers, dozens of experiments, and thousands of run records, far too much to serialize into any practical context window. Even recent context-window extensions and per-turn summarization compress this history lossily and asymmetrically: the model cannot programmatically re-inspect the parts it needs, only the parts a prior summarizer chose to preserve. The agent must reason *over* the workspace, not carry it in the prompt, or it loses focus and drifts off-program.

(ii) Calibration and reproducibility. An unsupervised agent commits to no prior expectation, so any outcome can be rationalized post-hoc as what we call *agentic p-hacking*: the agent runs the post-hoc selection pipeline that pre-registration was invented to prevent. Without a content-addressed prediction recorded *before* the run, the trace cannot be reproduced or verified.

(iii) Human-in-the-loop steerability. Research is iterative: each finding reshapes the next question. Agents must work *for* the researcher, redirectable mid-flight when findings warrant a pivot, and aligned with the evolving question rather than the one originally asked.

This paper's central claim: **modern CLI agents are well-calibrated autonomous ML researchers**, when paired with a platform that resolves the three challenges above. We deliver this through **Distillate**, a principled multi-agent autonomous research steering and orchestration harness that

addresses all three challenges (Figures 1–3). An **Orchestrator** RLM Agent acts as a co-scientist in a chat surface, surfacing literature, comparing past runs, and designing the research program with the researcher. It launches a swarm of autonomous **Experimentalist** RLM Agents (one per experiment) that execute the program without supervision, communicating with the Orchestrator through a content-addressed filesystem (`.distillate/`) and tool contracts (`start_run`, `conclude_run`) that gate training launches on a cryptographically committed pre-registration.

Contributions. Three architectural commitments answer the three challenges, plus one empirical contribution.

(1) Recursive Language Model agents over a research workspace (addresses coherence, Section 3). Both the Orchestrator and the Experimentalists are RLMs [Zhang et al., 2025]: they reason *over* a typed Python workspace (`lab.*` for the Orchestrator, `xp.*` for the Experimentalist) programmatically, rather than serializing the workspace into the prompt. The agent's state is bound as native Python variables it queries on demand, with delegated work spawned to sub-LLM agents in isolated contexts.

(2) Cryptographic pre-registration linking intent to outcome (addresses calibration and reproducibility, Section 3). A tool contract gates training: before any script may launch, the Experimentalist calls `start_run` with a falsifiable numeric prediction and self-rated confidence, pushed to a Git history that traces the experimental path. A two-commit protocol content-addresses the prediction in the run's witness ancestry, making the trace forensically verifiable and foreclosing in-context p-hacking [Lakens, 2022].

(3) Human-in-the-loop Orchestrator + Experimentalist swarm (addresses steerability and alignment, Section 3). The Orchestrator translates the researcher's intent into autonomous runs and surrenders the workspace until next consulted; a bidirectional steering channel (Figure 1) gives the researcher a low-overhead path to redirect a stalled Experimentalist mid-flight without restarting the run.

(4) Empirical demonstration of autonomous ML research, epistemic calibration, and head-to-head ELO ranking (Sections 4–5). To our knowledge, we provide the first systematic measurement of whether autonomous coding agents' pre-registered predictions correlate with realized task success. Across three scientific domains — combinatorial mathematics (Tiny-MatMul), molecular biology (GlycoGen), and computer vision (CIFAR-100) — and three frontier coding agents (k=9 campaigns, n=326 paired runs), pre-registered improvement $\Delta\hat{y}$ predicts realized improvement Δy at Pearson $r = +0.35$ (cluster-robust 95% CI $[+0.23, +0.49]$, $p < 0.001$), four times stronger than a frozen initial-belief baseline ($r = +0.09$, n.s.). We rank the three agent families head-to-head via Sequential ELO, Bradley–Terry MLE, and a three-axis composite research score ($S_{\text{peak}}, S_{\text{eff}}, S_{\text{prereg}}$), finding that Claude Code Sonnet 4.6 leads on both model quality and forecast calibration while self-rated confidence c is model-specific across vendors. Most importantly, the **GlycoGen Experimentalist autonomously reached state-of-the-art on de novo glycan structure prediction** (92.5% vs. 80.9% prior leak-free SOTA) [Lacombe, 2026].

2 Related Work

Recursive language models, tool use, and language-model programs. The term *Recursive Language Model* is due to Zhang et al. [2025], who treat a long prompt as data within a Python REPL and let the root model programmatically inspect, decompose, and recursively spawn sub-LLM calls over slices of that prompt. Their RLM processes inputs roughly two orders of magnitude beyond the model's nominal context window. The companion *Mismanaged Geniuses Hypothesis* [Zhang et al., 2026] argues that frontier LMs are severely underutilized because the systems managing them rely on brittle, human-engineered task decomposition: the next leap in capability may come from giving the model a richer space of decompositions rather than from further scaling. We take that prescription seriously: Distillate instantiates both the Orchestrator and the Experimentalists as RLMs over typed Python namespaces (Section 3), so the "management" of the LM is itself programmatic, principled, and auditable rather than ad-hoc scaffolding. The proposal sits at the end of a longer arc through tool-augmented and program-shaped language model use: **ReAct** [Yao et al., 2023b] interleaves reasoning with tool actions; **Toolformer** [Schick et al., 2023] self-supervises tool invocation; **Tree of Thoughts** [Yao et al., 2023a] and **Self-Refine** [Madaan et al., 2023] treat the search over reasoning steps and self-critique as the unit of computation; **HuggingGPT** [Shen et al., 2023] and **Gorilla** [Patil

et al., 2023] cast the language model as a controller over external services; and **DSPy** [Khattab et al., 2024] gives the program-as-LM-pipeline idea a compiler. None of these works addresses the case where the program must persist across sessions and operate over a workspace whose total size exceeds any single context window. This is the regime Distillate addresses.

Long-horizon agents and autonomous research. Long-horizon agent behavior is evaluated by benchmarks (**GAIA** [Mialon et al., 2023], **AgentBench** [Liu et al., 2024], **WebArena** [Zhou et al., 2024], METR's **RE-Bench** [Wijk et al., 2024], **SWE-bench** [Jimenez et al., 2024, Cognition AI, 2024, Deng et al., 2025], **MLE-bench** [Chan et al., 2024]) and addressed by multi-agent frameworks such as **Magentic-One** [Fourney et al., 2024] and **AutoGen** [Wu et al., 2023]. Applied to science: **ChemCrow** [Bran et al., 2024] and **Coscientist** [Boiko et al., 2023] embed LLMs in chemistry automation; the **Virtual Lab** [Swanson et al., 2024, Gao et al., 2024] composed agent personas to design SARS-CoV-2 nanobody binders; Sakana's **AI Scientist** [Lu et al., 2024, Yamada et al., 2025] (see also Beel et al. 2025 for an external evaluation) and **Agent Laboratory** [Schmidgall et al., 2025] target the full pipeline; **Kosmos** [Edison Scientific, 2025] runs 12-hour discovery cycles with a shared world model; **AIDE** [Jiang et al., 2025] frames ML engineering as code optimization via tree search; **MLR-Copilot** [Li et al., 2024] stages idea generation, implementation, and execution as separate sub-agents; **POPPER** [Huang et al., 2025] casts hypothesis falsification as the task; **FunSearch** [Romera-Paredes et al., 2024] and **AlphaEvolve** [Novikov et al., 2025] embed LLMs in evolutionary loops. A public auto-research culture (Karpathy's open projects [Karpathy, 2022–2025], the **AdderBoard** [Lee et al., 2023, Papailiopoulos and contributors, 2024]) conducts small fully-traced experiments in the spirit Distillate aims to make routine. *None of these systems measure whether the agent's numerical beliefs about its own upcoming actions are calibrated to outcomes.* The Kosmos authors themselves identify the absence of "calibrated confidence levels per claim" as future work. The common thread is enthusiasm for full automation; we instead treat the human as the source of intent, the agent as a measurable epistemic actor, and pre-registration as the instrument that makes its forecasts falsifiable.

Self-improving agents, durable memory, and orchestration protocols. **Reflexion** [Shinn et al., 2023] introduced verbal reinforcement learning; **Voyager** [Wang et al., 2023] an open-ended skill library; **Generative Agents** [Park et al., 2023] reflective memory streams; **MemGPT** [Packer et al., 2023] formalizes hierarchical memory. All share the conviction that productive autonomy at extended horizons requires durable, agent-authored memory; all keep that memory internal to the agent. Distillate externalizes it to a filesystem protocol that survives process boundaries, permitting an experiment to be paused, resumed under a different harness, and inspected by both user and shell. Hierarchical orchestration is now standard [Wu et al., 2023] (LangGraph, CrewAI, AutoGen); the **Model Context Protocol** [Anthropic, 2024] makes Distillate's Experimentalist backends interchangeable. We depart from the orchestration mainstream in insisting that long-running subprocess agents *not* be supervised during execution, since polling is hostile to multi-day timescales.

LLM confidence, pre-registration, and pairwise agent rating. LLMs frequently know more than they verbalize about their own uncertainty [Kadavath et al., 2022, Lin et al., 2022], yet remain miscalibrated under realistic prompts: **ClimateX** [Lacombe et al., 2023] finds systematic overconfidence on expert-graded scientific statements; **Don't Think Twice** [Lacombe et al., 2025] shows that extended reasoning budgets *worsen* calibration on reasoning LLMs, a counterpoint to test-time scaling. Both measure calibration on fixed text instances against external ground truth. Our pre-registration substrate lifts the question to *agent self-forecasting under autonomous execution*, where the prediction concerns a training run the agent is about to conduct and the agent's own choices determine the outcome (Section 5). Originating in the social and clinical sciences as a remedy for p-hacking and HARK-ing [Lakens, 2022], pre-registration was adapted to predictive modeling by Hofman et al. [2023] amidst an ML reproducibility crisis [Kapoor and Narayanan, 2023, Semmelrock et al., 2025, Huntington-Klein et al., 2025]; Distillate enforces it structurally via the agent's tool surface (Table 1). Finally, to compare heterogeneous agents head-to-head across research tasks with incompatible raw metrics, we adapt the Bradley–Terry maximum-likelihood framework [Bradley and Terry, 1952] popularized by **Chatbot Arena** [Chiang et al., 2024] from single-turn human preference to multi-day autonomous research outcomes.

Table 1: The `start_run()` pre-registration schema. All seven fields are required before any training process can be launched. The tool returns `(run_id, run_number)`, writes the entry to `runs.jsonl`, and commits and pushes the `prereg:` anchor to `origin` before training begins.

Field	Type	Description
`description`	string	The architectural or algorithmic approach being tried this run
`hypothesis`	string	Mechanistic explanation for why the change should work
`prediction`	string	Concrete, falsifiable qualitative outcome statement
`predicted_metric`	string	Name of the primary optimization metric (e.g. `val_accuracy`)
`predicted_value`	float	Pre-registered numeric point forecast $\hat{y}_n \in \mathbb{R}$
`confidence`	int	Self-rated epistemic confidence score $c_n \in [0, 100]$
`rationale`	string	Supporting evidence from prior runs in `runs.jsonl` or literature

3 The Distillate Harness

Distillate's two-agent architecture (Figures 1–3) realizes the three architectural commitments at the system level within an Integrated Agents Environment (IAE). The **Orchestrator** (named to the user as Nicolas, after the alchemist Nicolas Flamel [Ball, 2025]) is a persistent in-process agent on the Claude Agent SDK with $\sim$96 MCP tools spanning papers, experiments, projects, sessions, and runs; it acts as a co-scientist in a chat surface, surfacing literature from Zotero and arXiv, comparing past runs, and launching or steering experiments. **Experimentalists** are CLI agent sessions running in `tmux`, one per experiment, that hill-climb against an objective metric without supervision. Through a uniform `HarnessAdapter` protocol and MCP tool contract, Claude Code, OpenAI Codex CLI, Gemini CLI, and OpenHands serve as interchangeable Experimentalist backends over filesystem-only state.

3.1 RLM agents over a research workspace (coherence)

Both agents run inside a sandboxed Python REPL with a typed namespace (`lab.*` for the Orchestrator, `xp.*` for the Experimentalist) restricted to the workspace API (no arbitrary `import`/`eval`/`exec`). The workspace is bound as native Python objects the agent queries on demand: expressions such as `lab.experiments.recent(5).runs.peak_metric()` return fluent collections in-process, lifting the Recursive Language Model paradigm [Zhang et al., 2025] from inference-time prompt slicing to research-time workspace navigation. Sub-LLM delegation follows an explicit three-tier cost ladder from \$0 (direct Python namespace inspection) through `llm_query()` (a fast Haiku sub-call) to `delegate()` (an isolated sub-agent with a domain-scoped tool subset, such as the *Apothecary* literature scout); a per-experiment ledger (`.distillate/cost.json`) is atomically updated on every sub-LM call so the agent can self-budget across a multi-day campaign.

3.2 Pre-registration as a cryptographic intent-to-outcome link (calibration, reproducibility)

Before any training script may launch, the Experimentalist must call `start_run` with the seven-field schema in Table 1, and the `run_id` it returns is the only valid identifier for any subsequent training-related call. The schema records the agent's full epistemic state at every run: *what* it is trying, *why* it expects the mechanism to work, *how much* improvement it anticipates ($\hat{y}_n$), and *how confident* it is (c_n). Moreover, each experiment enforces a strict *1 Agent, N Runs* attribution invariant: a campaign directory is bound to a single `(agent_id, harness_id)` pair across all N sequential runs so that downstream calibration and ELO ratings are cleanly attributable.

After training completes (or is terminated by `distillate-run` when the wall-clock deadline in `.distillate/budget.json` is reached), `conclude_run` writes the realized results, an agent-authored subjective verdict (`confirmed` / `refuted` / `inconclusive`), and a Reflexion-style [Shinn et al., 2023] `belief_update` synthesis that is materialized into `.distillate/context.md` for the next run. Downstream calibration analysis uses a separate objective *analysis verdict* $v_n = \mathbb{1}[|y_n - \hat{y}_n| \leq \tau]$ with $\tau = 0.10$ (Section 5). Both calls write to an append-only `runs.jsonl`, with each pre-registration row paired to its conclusion row by `run_id`. We thus implement pre-registration [Lakens, 2022, Hofman et al., 2023] structurally as a precondition for compute rather than as an honor-system discipline.

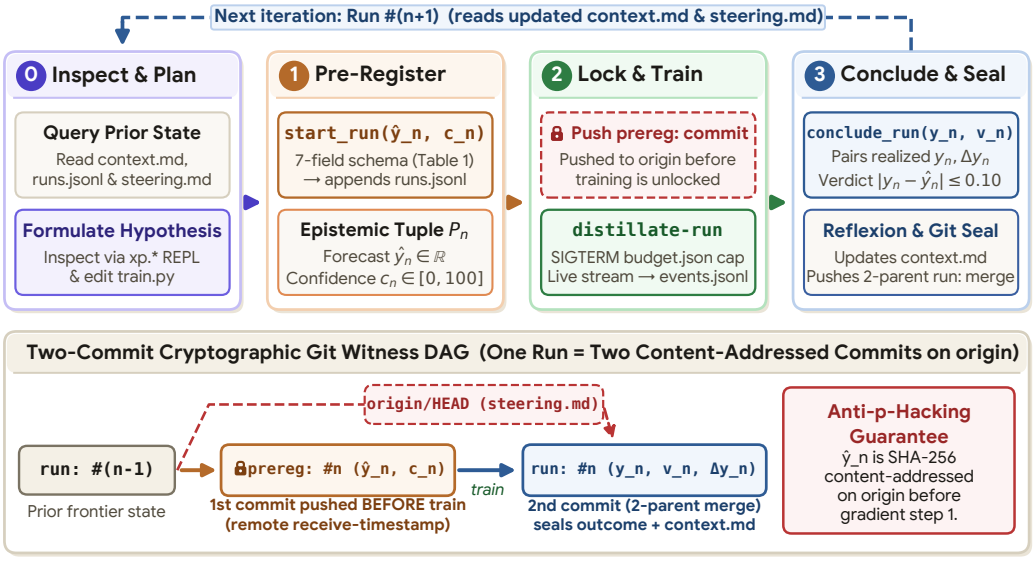


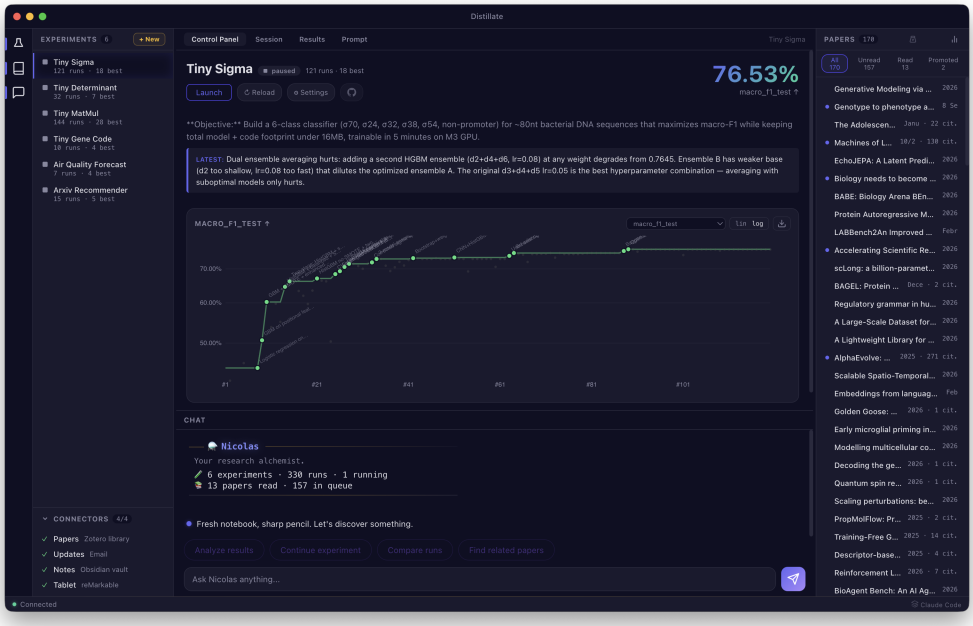
Figure 2: **The cryptographic pre-registration and auto-research loop inside one Experimentalist run.** Each iteration executes a four-stage protocol: (0) inspect prior workspace state and append a seven-field pre-registration tuple $(\hat{y}_n, c_n)$ via `start_run()`; (1) commit and push the `prereg:` commit to `origin` before training is unlocked; (2) launch budget-enforced training via `distillate-run` while streaming live metrics to `events.jsonl`; and (3) record realized y_n, objective verdict v_n, Reflexion synthesis in `context.md`, and seal the two-parent `run:` merge commit via `conclude_run()`.

The pre-registration commit is anchored on `origin` as *one run = two Git commits* (Figure 2). The `prereg:` commit stages the prereg row and is pushed *before* training launches; the `run:` commit, emitted by `conclude_run`, is a two-parent merge commit whose ancestry incorporates anything the Orchestrator pushed during training. Once the prereg is pushed, the prediction cannot be modified without breaking the commit hash chain, the remote stamps an arrival time outside the agent's control, and a third-party reader can walk the Git history to verify exactly what evidence the agent saw before each run and that no prediction was silently rewritten. For each run n, the pre-registration tuple is $P_n = (m_n, \hat{y}_n, c_n)$ with m_n the predicted metric, $\hat{y}_n \in \mathbb{R}$ the predicted value, and $c_n \in \{0, \ldots, 100\}$ the self-rated confidence; the analysis dataset is $\mathcal{D} = \{(P_n, y_n, v_n)\}_{n=1}^{N}$ with y_n the observed value of m_n and v_n the objective binary verdict.

3.3 Human-in-the-loop Orchestrator and Experimentalist swarm (steerability, alignment)

Mainstream multi-agent orchestration [Wu et al., 2023] polls workers from the orchestrator, a pattern hostile to multi-day timescales. Distillate disengages instead: the Orchestrator launches the Experimentalist and surrenders the workspace until next consulted (Figure 3). The pre-registration trace then makes the trigger condition for human intervention well-defined — consecutive prereg/outcome rows with persistent prediction error and no belief update — and the Orchestrator's steering channel (Figure 1) lets the researcher inject a single corrective hint into `.distillate/steering.md` on the next `start_run` without restarting the session. While coding CLIs already permit interactive user interruption, Distillate makes the *intervention trigger* quantitatively legible from `runs.jsonl` (Section 5 details the Gemini Flash unstuck-by-one-hint episode).

Open-source release and reproducibility. Distillate is released as an open-source package at https://github.com/rlacombe/distillate, including the desktop IAE, the harness adapters, the MCP tool contracts, the raw `runs.jsonl` traces for all nine campaigns in Section 4, and the analysis scripts that reproduce every table and figure in this paper. System prompts and the `.distillate/` repository layout are documented in Appendices A–B.



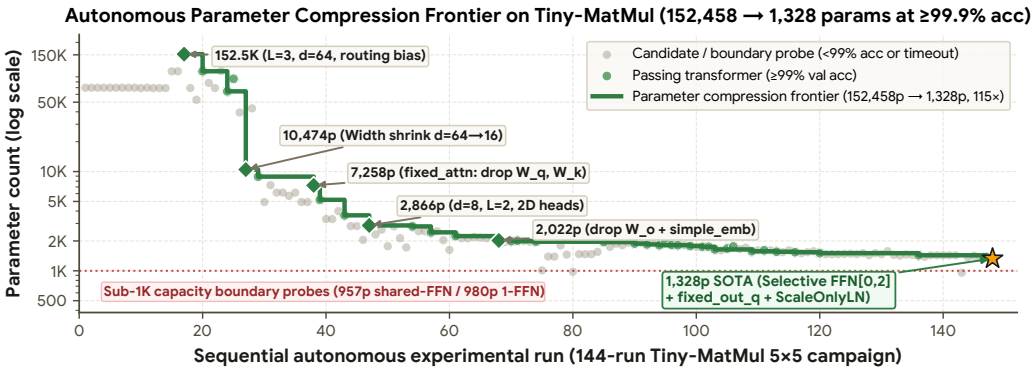


Figure 3: **The shipping Distillate Integrated Agents Environment (IAE). Top:** Desktop environment showing the experiment sidebar and Zotero/Obsidian/reMarkable connectors (left), the live `Tiny Sigma` campaign dashboard at 76.53% macro-F_1 across 121 runs with its latest Reflexion belief update and annotated frontier curve (upper center), the persistent conversational shell with the Orchestrator **Nicolas** managing 6 experiments and 330 runs alongside 170 tracked papers (lower center), and the integrated literature queue (right). **Bottom:** Vector parameter-compression frontier from the 144-run `Tiny-MatMul` (5×5) campaign, showing all candidate runs (slate points), sub-1K boundary probes (957p–980p, amber diamonds), and the monotonically decreasing passing frontier ($\geq 99\%$ accuracy, emerald step curve with autonomous architectural milestone callouts) compressing a passing transformer by $115\times$ from $152{,}458$ parameters down to $1{,}328$ parameters (99.97% accuracy).

4 Experiments

To test whether autonomous research agents can be trusted with multi-day scientific work, we deploy Distillate across three experimental domains chosen to span scientific surfaces with different ground truths: *combinatorial mathematics* (a closed analytic problem where the agent satisfices on accuracy under a tight compute budget while minimizing model size), *molecular biology* (an open scientific problem with no analytic solution, only experimental mass-spectrometry ground truth), and *computer vision* (a canonical benchmark where the agent hill-climbs an accuracy ceiling under strict parameter and wall-clock caps). All campaigns are driven through Distillate and pre-registered run-by-run

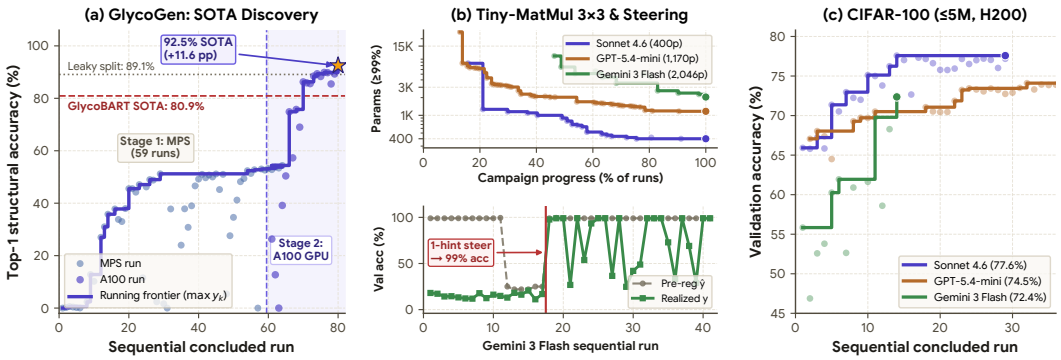


Figure 4: **Autonomous research trajectories across three scientific domains. (a) GlycoGen:** Across a two-stage campaign (59 concluded runs on Apple Silicon MPS followed by 21 scaling runs on an A100 GPU), the Distillate Experimentalist autonomously discovers a 50M-parameter discrete flow matching architecture that reaches 92.5% top-1 structural accuracy, surpassing the leak-free GlycoBART SOTA (80.9%) by $+11.6$ pp and exceeding the previously published leaky split (89.1%). **(b) Tiny-MatMul** 3×3**:** Top: parameter compression frontiers ($\geq 99\%$ token accuracy) across three CLI agents, converging to non-canonical architectures at 400 (Sonnet 4.6), 1,170 (GPT-5.4-mini), and 2,046 parameters (Gemini 3 Flash). Bottom: pre-registered forecast $\hat{y}$ versus realized accuracy y for Gemini 3 Flash; ten consecutive runs at the $\sim$20% mod-2 floor with persistent $\hat{y}\approx99\%$ make the stall legible in `runs.jsonl`, and a single Orchestrator hint in `steering.md` immediately unlocks 99% accuracy. **(c) CIFAR-100 Speedrun:** Validation accuracy frontiers under a $\leq$ 5M-parameter, 5-min H200 cap (77.6% Sonnet 4.6, 74.5% GPT-5.4-mini, 72.4% Gemini 3 Flash).

(Figure 4). Tiny-MatMul and CIFAR-100 are run across all three coding agents (Claude Code, Codex CLI, Gemini CLI); GlycoGen is a deep multi-day single-agent campaign on Claude Code.

Combinatorial mathematics: Tiny-MatMul. Smallest transformer that solves $N\times N$ mod-10 matrix multiplication under a 5-min wall-clock training budget; the agent satisfices on accuracy (≥ 0.99) while minimizing parameter count (Figure 3, bottom; Figure 4b). The 5×5 deployment is single-agent (Sonnet 4.6 on Claude Code, Apple Silicon MPS); the 3×3 deployment is a three-vendor head-to-head across Sonnet-class agents (Sonnet 4.6 on Claude Code, GPT-5.4-mini on Codex CLI, Gemini 3 Flash on Gemini CLI), running $\sim$6 h of unsupervised wall-clock per agent. Pooled across all four deployments, $n_{\text{mm}} = 203$ on-metric pairs.

Molecular biology: GlycoGen. De novo glycan structure prediction from tandem mass spectrometry under the continuous-time discrete flow matching family. This single-agent campaign (Sonnet 4.6 on Claude Code) was deployed at two rungs of the scaling ladder (Figure 4a): 80 runs on Apple Silicon (MPS, $\sim$23 h active compute; 59 concluded on-metric runs) and 21 scaling runs on an NVIDIA A100 GPU (101 runs total). Primary metric throughout is `top1_structural_accuracy`; after pairing pre-registrations to `conclude_run` rows and excluding runs from a mid-experiment metric exploration phase, $n_{\text{glyco}} = 49$ delta pairs. **The Experimentalist's autonomous campaign reached state-of-the-art on de novo glycan structure prediction**; the discovered architecture and scaling ablations are verified independently in our companion paper [Lacombe, 2026], demonstrating that autonomous agents can conduct ML research at the frontier of a scientific discipline.

Computer vision: CIFAR-100 [Krizhevsky, 2009]. Three-vendor head-to-head experiment on dedicated NVIDIA H200 GPUs via Hugging Face Jobs (Figure 4c) — the same three frontier agents as Tiny-MatMul 3×3 on research-grade GPUs, accuracy-maximization protocol, $\leq$ 5M-parameter classifier cap, 5-min per-run and 6-hour total training budget, fixed 45K/5K train/val split with the 10K test set held out. Agents hill-climb and pre-register forecasts on validation accuracy, yielding $n_{\text{cifar}} = 74$ on-metric pairs.

The combined on-metric dataset $\mathcal{D}$ comprises $n = 326$ prediction–outcome delta pairs across $k = 9$ sequential campaigns.

Analysis formalism: epistemic calibration and composite ELO rating. We compute deltas relative to the running maximum within each campaign:

$$\Delta\hat{y}_n = \hat{y}_n - \max_{k<n} y_k, \qquad \Delta y_n = y_n - \max_{k<n} y_k. \tag{1}$$

The deployment question *"how much will this run push the frontier?"* reduces to the magnitude correlation $r_1 = r(\Delta\hat{y}, \Delta y)$ on the pooled Δ-pairs. Working with deltas rather than raw $(\hat{y}, y)$ levels absorbs two confounds that otherwise inflate raw correlations: (i) a within-campaign time trend on accuracy-maximization tasks (predictions and outcomes both rise as architectures improve), and (ii) a saturation artifact on satisficing tasks (where most predictions cluster near the accuracy gate). Significance and confidence intervals come from a cluster-robust bootstrap that resamples whole campaigns (B=5,000 resamples with replacement, fixed seed).

The self-rated confidence channel is evaluated by three metrics: (1) the raw correlation $r_2 = r(c, \Delta y)$ on delta-pairs; (2) the sign-corrected correlation $r_2^\star = r(c,\, \mathrm{sign}(\Delta\hat{y})\cdot\Delta y)$, which credits confident, accurate predictions of performance declines ($\Delta\hat{y} < 0, \Delta y < 0$) as directional hits rather than anti-correlation; and (3) on the verdict dataset, the binned Expected Calibration Error

$$\mathrm{ECE} = \sum_{b=1}^{B} \frac{|\mathcal{B}_b|}{N} \left|\mathrm{acc}(\mathcal{B}_b) - \mathrm{conf}(\mathcal{B}_b)\right|, \tag{2}$$

with confidence bins $\mathcal{B}_b$ of width 10 over $[0, 100]$ and objective confirmation verdict $|y_n - \hat{y}_n| \le 0.10$.

Finally, to evaluate and rank autonomous research agents across multi-run campaigns, we combine outcome quality, sample efficiency, and pre-registration calibration into the **Distillate composite research score**:

$$S(a,t) = 0.50\, S_{\mathrm{peak}}(a,t) + 0.25\, S_{\mathrm{eff}}(a,t) + 0.25\, S_{\mathrm{prereg}}(a,t), \tag{3}$$

where $S_{\mathrm{peak}} = \mathrm{clip}\left(\frac{m_{\mathrm{best}} - m_{\mathrm{base}}}{m_{\mathrm{ref}} - m_{\mathrm{base}}}, 0, 1\right)$ is normalized peak metric progress, $S_{\mathrm{eff}} = 1 - \frac{n_{0.8}}{N_{\max}}$ measures how rapidly the agent reaches 80% of its peak improvement, and $S_{\mathrm{prereg}} = \frac{1}{|R^*|}\sum_{n\in R^*} \max\left(0, 1 - \frac{|\hat{y}_n - y_n|}{m_{\mathrm{ref}} - m_{\mathrm{base}}}\right)$ rewards calibrated pre-registered forecasts across runs. Pairwise outcomes across tasks are aggregated into agent ratings via both Sequential ELO (K=32, with B=5,000 match-bootstrap 95% CIs) and Bradley–Terry maximum-likelihood estimation [Bradley and Terry, 1952, Chiang et al., 2024].

5 Results

To our knowledge, this is the first systematic measurement of whether autonomous coding agents' pre-registered predictions correlate with realized task success on multi-day research campaigns. Prior work measures calibration on fixed text instances against external ground truth [Lacombe et al., 2023, 2025]; here we lift the question to agent self-forecasting under autonomous execution, where the prediction concerns a training run the agent is about to conduct, with no fixed ground truth and the additional complication that the agent's own choices determine the outcome. Two findings stand out: $\Delta\hat{y}$ carries deployable signal across all three models, while the self-rated-confidence channel c *flips sign* between them — breaking the assumption that LLM self-confidence is a portable scalar.

Headline: predicted improvement tracks realized improvement. Pooled across all nine campaigns and n=326 runs spanning three scientific domains and three frontier coding agents, pre-registered improvement $\Delta\hat{y}$ tracks realized improvement Δy at Pearson r=+0.35 (cluster-robust 95% CI $[+0.23, +0.49]$, p<0.001; Figure 5a). Autonomous in-context descent beats random; the agent's own numerical forecast about a run it has not yet executed carries deployable signal. Within-campaign effect sizes are strongest on deep scientific and architectural search — r=+0.59 on GlycoGen (n=49, $p<10^{-6}$) and r=+0.40 pooled across Tiny-MatMul (n=203, $p<10^{-8}$) — and positive on 5-minute CIFAR-100 speedruns (r=+0.28 for Sonnet, n=28, p=0.15). To verify that this signal reflects active belief updating across runs rather than running-maximum drift, we evaluate three null baselines on the full n=326 dataset (Figure 5a, inset): a within-campaign permutation null (B=5,000 shuffles) yields r=+0.01 (95% CI $[-0.11, +0.13]$); a frozen-first-prediction baseline ($\hat{y}_n = \hat{y}_1$ held fixed after run 1) yields r=+0.09 (p=0.12, cluster-p=0.40, n.s.); and a constant-mean predictor yields r=0. Active in-context belief updating thus accounts for roughly three-quarters of the agent's forecasting power.

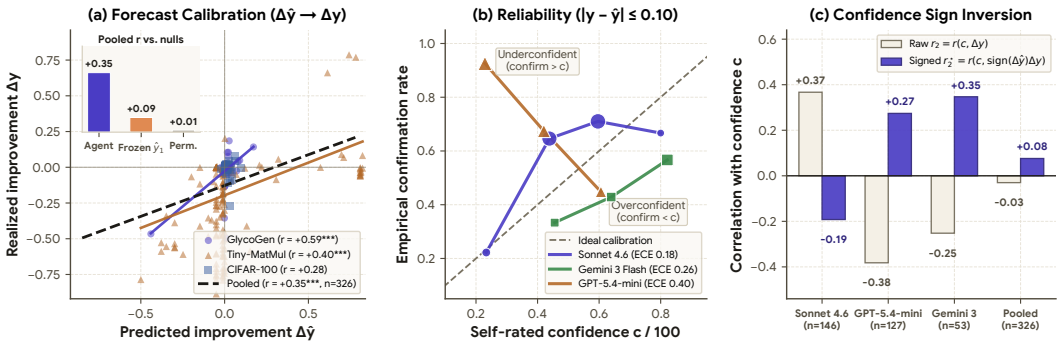


Figure 5: **Epistemic self-forecasting and confidence calibration across n=326 pre-registered runs (k=9 campaigns). (a) Pre-registered forecast vs. realized frontier delta:** Predicted improvement $\Delta\hat{y}$ significantly tracks realized improvement Δy (r=+0.35, cluster-robust p=0.001). Inset: active agent belief updating outperforms a frozen-first-prediction baseline ($\hat{y}_n = \hat{y}_1$, r=+0.09, n.s.) by 4× and a within-campaign permutation null (r=+0.01). **(b) Reliability diagram:** Empirical confirmation rate ($|y_n - \hat{y}_n| \leq 0.10$) versus self-rated confidence $c/100$; Sonnet 4.6 is best calibrated overall (ECE=0.18), whereas Gemini 3 Flash is systematically overconfident at high confidence bins and GPT-5.4-mini exhibits a non-monotonic profile (ECE=0.40). **(c) Confidence channel sign inversion:** Raw $r_2 = r(c, \Delta y)$ flips sign against sign-corrected $r_2^\star = r(c, \text{sign}(\Delta\hat{y})\Delta y)$ across every vendor, while pooled correlations remain near zero (−0.03 raw, +0.08 signed), confirming that scalar self-confidence is model-specific rather than portable.

Within-Claude: Sonnet 4.6 across the three experiments. Table 2 reports four signals per experiment for the within-Claude deployments: the magnitude correlation $r_1 = r(\Delta\hat{y}, \Delta y)$, the confidence-to-improvement correlation $r_2 = r(c, \Delta y)$, the frontier-push hit rate $P(\Delta y{>}0 \mid \Delta\hat{y}{>}0)$ on the frontier-push subset $\mathcal{F}$, and binned ECE under $|y - \hat{y}| \leq 0.10$. The magnitude signal is highly significant on GlycoGen (r_1=+0.59, $p<10^{-6}$) and Tiny-MatMul (r_1=+0.54, $p<10^{-6}$), and positive on CIFAR-100 (r_1=+0.28, n=28). ECE is lowest on Tiny-MatMul (0.19) and pooled across all Sonnet runs reaches 0.18 (Figure 5b).

Table 2: Within-Claude (Sonnet 4.6 / Claude Code) calibration signals across the three experiments. $r_1 = r(\Delta\hat{y}, \Delta y)$ Pearson; $r_2 = r(c, \Delta y)$ Pearson; hit rate is $P(\Delta y{>}0 \mid \Delta\hat{y}{>}0)$ on the frontier-push subset of size $|\mathcal{F}|$; ECE binned at width 10 over $[0, 100]$ with verdict $|y - \hat{y}| \leq 0.10$.

Experiment	n	r_1	p	r_2	p	$\lvert\mathcal{F}\rvert$	hit	ECE
GlycoGen	49	+0.59	$< 10^{-6}$	+0.24	0.10	41	49%	0.36
Tiny-MatMul	69	+0.54	$< 10^{-6}$	+0.40	$< 10^{-4}$	12	67%	0.19
CIFAR-100*	28	+0.28	0.15	+0.18	0.35	28	22%	0.52

* Sonnet slice of the cross-model H200 deployment; see Table 3 for the full cross-model breakdown.

Cross-model replication and the confidence sign inversion. The magnitude signal $\Delta\hat{y}{\to}\Delta y$ replicates across all three frontier coding agents on Tiny-MatMul (r_1=+0.54 Sonnet 4.6, r_1=+0.39 GPT-5.4-mini, r_1=+0.38 Gemini 3 Flash; all $p \leq 0.02$) and is directionally consistent across all three on CIFAR-100 (Table 3). By contrast, the self-rated confidence channel c does not replicate as a portable scalar: under the raw metric $r_2 = r(c, \Delta y)$, Codex and Gemini exhibit negative correlations (−0.28 and −0.30 on Tiny-MatMul) because both agents frequently pre-register high confidence on runs they accurately expect to regress below the running frontier ($|\mathcal{F}|$=3 of 94 for Codex). When we apply the sign-corrected formulation $r_2^\star = r(c, \text{sign}(\Delta\hat{y}){\cdot}\Delta y)$ across the complete n=326 dataset (Table 4 and Figure 5c), the sign of every vendor's correlation reverses (+0.37 → −0.19 for Sonnet; −0.38 → +0.27 for GPT-5.4-mini; −0.25 → +0.35 for Gemini Flash), while the grand pooled correlation remains near zero under both metrics (−0.03 raw, +0.08 signed). Point-forecast deltas $\Delta\hat{y}$ are thus portable across model families out of the box, whereas scalar confidence c must be calibrated per model family.

Table 3: Per-model breakdown on the two cross-model experiments. Same column conventions as Table 2. The $\Delta\hat{y}\to\Delta y$ channel replicates across models; the $c\to\Delta y$ channel is model-specific in sign and significance.

Experiment	Model	n	r_1	p	r_2	p	$\lvert\mathcal{F}\rvert$	hit	ECE
Tiny-MatMul	Sonnet 4.6	69	$+0.54$	$< 10^{-6}$	$+0.40$	$< 10^{-4}$	12	67%	0.19
	GPT-5.4-mini	94	$+0.39$	$< 10^{-4}$	-0.28	0.006	3	33%	0.28
	Gemini 3 Flash	40	$+0.38$	0.02	-0.30	0.06	18	22%	0.30
CIFAR-100*	Sonnet 4.6	28	$+0.28$	0.15	$+0.18$	0.35	28	22%	0.52
	GPT-5.4-mini	33	$+0.23$	0.19	$+0.05$	0.78	33	33%	0.74
	Gemini 3 Flash	13	$+0.21$	0.49	$+0.26$	0.39	13	–	0.17

*Cross-model CIFAR-100 deployment on dedicated H200 GPUs via Hugging Face Jobs.

Table 4: Raw vs. signed confidence-channel statistic across the complete dataset (n=326, all 9 campaigns). Signed $r_2^\star = r(c, \text{sign}(\Delta\hat{y})\cdot\Delta y)$ removes the raw metric's confound on decline predictions. The vendor sign pattern reverses between the two formulations; both yield pooled correlations near zero.

Model	n	raw r_2	signed $r_2^\star$	difference
Claude Code Sonnet 4.6	146	$+0.37$	-0.19	-0.56
Codex GPT-5.4-mini	127	-0.38	$+0.27$	$+0.65$
Gemini 3 Flash	53	-0.25	$+0.35$	$+0.60$
Pooled	326	-0.03	$+0.08$	$+0.11$

Head-to-head: best model produced, composite research score, and ELO ranking. The campaigns also benchmark *what each agent built* (Figure 4b–c). On Tiny-MatMul 3×3 (smallest passing transformer at $\geq 99\%$ accuracy), the best architecture is Claude Code Sonnet 4.6's 400 params at 99.2%, Codex GPT-5.4-mini's 1,170 params at 99.6%, and Gemini 3 Flash's 2,046 params at 99.0% — a $\sim$5× parameter spread achieved through non-canonical low-rank factorizations and cross-layer weight sharing. On CIFAR-100 ($\leq$ 5M params, 5-min H200 cap), peak validation accuracy is Claude Code Sonnet 4.6 at 77.6%, Codex GPT-5.4-mini at 74.5%, and Gemini 3 Flash at 72.4%. The best-calibrated agent overall (Sonnet 4.6, pooled ECE=0.18) also produces the best final models on both tasks and achieves the highest composite research score $S(a,t) = 0.84$ versus 0.70 for Codex and 0.67 for Gemini (Figure 6b). Aggregating the six pairwise head-to-head outcomes (three per task across two tasks) yields an ELO-style ranking [Bradley and Terry, 1952, Chiang et al., 2024] (Table 5 and Figure 6a): the Claude–Gemini gap (1558 vs. 1442 Sequential ELO; 1898 vs. 1500 Bradley–Terry ELO) is statistically separable under match-bootstrap resampling, establishing a quantitative template for live, contamination-resistant leaderboards of autonomous research agents.

Table 5: Head-to-head model ranking from six pairwise outcomes (three per task × two tasks: smallest passing transformer on Tiny-MatMul 3×3, peak val accuracy on CIFAR-100) and mean composite auto-research score $S(a,t)$ (Eq. 3). Sequential ELO at K=32, with B=5,000 match-bootstrap 95% CIs. Bradley–Terry MLE with Laplace $+0.5$ smoothing (since Claude is undefeated), converted to ELO scale with Gemini as reference (1500).

Model	Composite $S(a,t)$	Sequential ELO	95% CI	Bradley–Terry ELO
Claude Code Sonnet 4.6	0.84	1558	$[1529, 1582]$	1898
Codex GPT-5.4-mini	0.70	1500	$[1447, 1552]$	1699
Gemini 3 Flash	0.67	1442	$[1418, 1473]$	1500

Human-in-the-loop steering unlocks attractors. Gemini 3 Flash on Tiny-MatMul 3×3 stalled across ten autonomous runs from two compounding causes (architectures too small to clear the gate; misread training-budget instructions) — both visible in `runs.jsonl` as predicted accuracy near 0.99 against an actual parity floor at $\sim$0.20, yet the agent did not update its pre-registrations (Figure 4b, bottom). A single corrective hint through Orchestrator steering broke the attractor: the

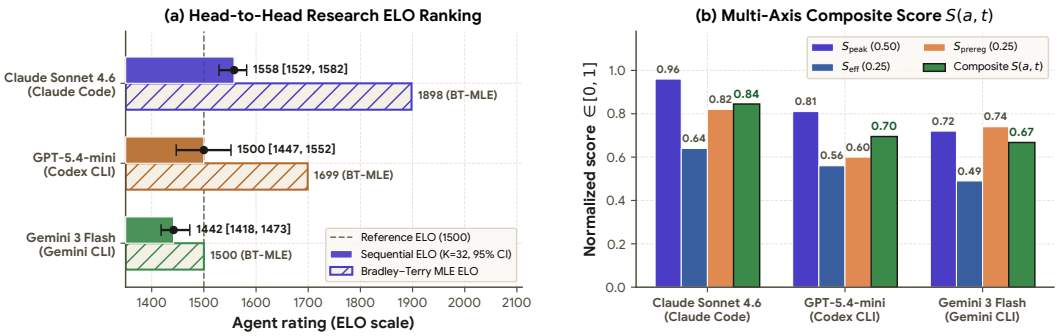


Figure 6: **Head-to-head autonomous research ranking and multi-axis score decomposition. (a) ELO & Bradley–Terry ratings:** Sequential ELO (K=32, solid bars with 95% match-bootstrap CIs) and Bradley–Terry MLE ELO (hatched bars, Laplace $+0.5$ smoothing) across the six pairwise head-to-head outcomes on Tiny-MatMul 3×3 and CIFAR-100. Claude Sonnet 4.6 is undefeated (1558 Sequential ELO, 1898 BT-MLE) and statistically separated from Gemini 3 Flash (1442 Sequential ELO). **(b) Composite score components:** Mean normalized peak performance (S_{peak}, weight 0.50), sample efficiency (S_{eff}, weight 0.25), pre-registration accuracy (S_{prereg}, weight 0.25), and composite score $S(a, t)$ across the cross-vendor campaigns.

next run trained a 10,200-parameter transformer for the full duration and hit 99% accuracy. The pre-registered trace makes intervention points *legible* — a researcher (or the Orchestrator on their behalf) scanning `runs.jsonl` sees the exact run where an agent stalls and can inject a corrective hint without restarting the campaign.

Case study: state-of-the-art on de novo glycan structure prediction. On the GlycoGen application [Lacombe, 2026], the Distillate loop progressed from 59 exploratory runs on Apple Silicon MPS to 21 pre-registered A100 scaling runs across five architectural dimensions (Figure 4a), converging on a 50M-parameter continuous-time discrete flow matching model that **exceeded the state of the art on de novo glycan structure prediction**: 92.5% (0.9251) top-1 structural accuracy at K=8 sampling versus 80.9% (0.8092) top-1 for GlycoBART [Abtheen et al., 2025] evaluated under the identical leak-free per-spectrum protocol (+11.6 pp improvement, and also surpassing the 89.1% originally reported under a leaky spectrum-level split with database snap-back). The autonomous loop also surfaced an empirical scaling regularity for this domain: training-step count dominates parameter count for sample efficiency. This is the strongest evidence yet for our thesis. We asked whether modern CLI agents are well-calibrated autonomous ML researchers; on this biomedical task, the answer is, unequivocally: **yes, they are**.

6 Discussion

Coherence: reasoning over a workspace works at long horizons. GlycoGen's 80-run Apple-Silicon campaign (∼23 h active compute) plus 21-run A100 scaling phase, and the Tiny-MatMul 5×5+3×3 campaigns (∼6 h per vendor per task, up to 144 runs in Figure 3), ran end-to-end without context exhaustion. Treating the research workspace as a typed Python object graph (`lab.*`, `xp.*`) and externalizing episodic memory into `.distillate/context.md` and `runs.jsonl` is the key architectural mechanism that keeps long-horizon autonomous research coherent over days of execution (Figure 7a).

Calibration: two trust signals, one portable and one model-specific. Pre-registration exposes two distinct epistemic signals: predicted improvement $\Delta\hat{y}$ and self-rated confidence c. Point-forecast improvement $\Delta\hat{y}$ replicates across all three model families (r=+0.35 pooled, p<0.001) and beats a frozen initial-belief baseline by fourfold; c does not, reversing sign across vendors between raw r_2 and sign-corrected $r_2^\star$ while pooling near zero. A researcher gets a usable $\Delta\hat{y}$ forecast out of the box for compute triage; scalar confidence c must be recalibrated empirically per agent family before deployment.

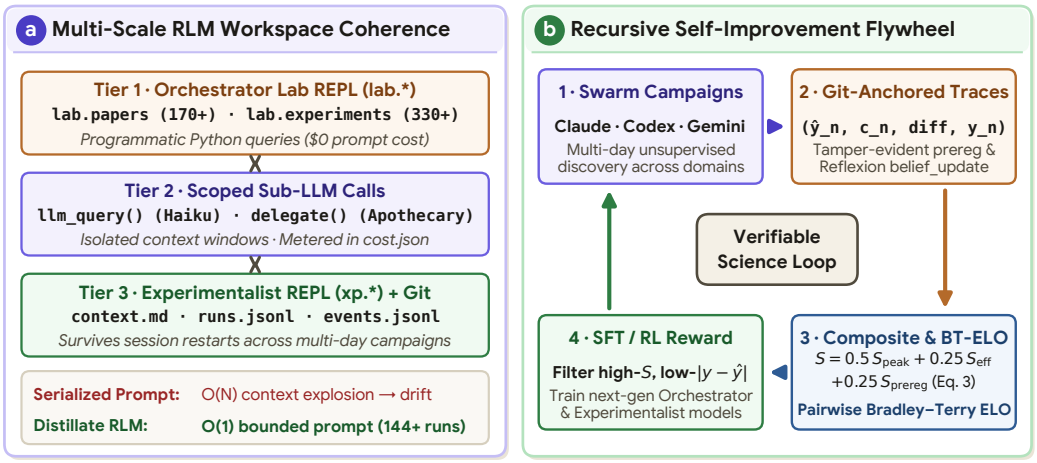


Figure 7: **Multi-scale RLM workspace coherence and the recursive self-improvement flywheel. (a)** Unlike traditional prompt-serialized loops whose context grows as $\mathcal{O}(N)$ until hitting context exhaustion (crimson dashed curve), Distillate maintains an $\mathcal{O}(1)$ bounded active prompt across 150+ autonomous runs (emerald line) by externalizing state into a 3-tier recursive workspace (`lab.*` Orchestrator REPL $\leftrightarrow$ `llm_query()`/`delegate()` sub-calls $\leftrightarrow$ `xp.*` Experimentalist REPL and `.distillate/context.md`, slate shaded area). **(b)** The four-stage recursive self-improvement loop: multi-day autonomous swarm campaigns (Stage 1) emit cryptographically anchored pre-registration traces on `origin` (Stage 2), which are scored via the composite benchmark metric $S(a,t)$ and Bradley–Terry ELO (Stage 3) to provide a dense, anti-p-hacked reward signal for RL/SFT distillation of next-generation Orchestrator and Experimentalist models (Stage 4).

Steerability and the recursive self-improvement loop. When an Experimentalist stalls, the pre-registration trace makes the divergence between belief ($\hat{y}_n$) and reality (y_n) immediately visible, allowing the Orchestrator's bidirectional steering channel to unstick the agent with a single hint (Figure 4b); the same substrate generalizes beyond ML to any domain where autonomous agents risk uncalibrated claims [Grundl, 2026]. Furthermore, because every Distillate campaign emits a cryptographically anchored trace of (hypothesis, $\hat{y}_n$, c_n, code diff, y_n, belief_update) scored by Eq. 3 and ranked by Bradley–Terry ELO (Figure 6), the benchmark naturally forms a recursive training environment (Figure 7b): high-composite traces with low calibration error provide a dense, verifiable reward signal for fine-tuning future Orchestrator and Experimentalist models on the task of conducting science itself.

7 Limitations and Future Work

Calibration scope and model tiers. The deepest multi-day campaign (GlycoGen) uses Sonnet 4.6 / Claude Code; cross-model experiments on Tiny-MatMul and CIFAR-100 use Sonnet-class agents under a 5-min per-run budget to keep multi-agent compute tractable. Expanding the head-to-head ELO matrix to flagship-tier models (Opus, GPT-5.4-pro, Gemini Ultra) across dozens of rotating community tasks will tighten the Bradley–Terry confidence intervals reported in Table 5.

Sandboxed pre-registration and component ablations. Pre-registration is enforced by MCP tool contract and attested by the two-commit Git hash chain; moving the `.distillate/` writer into an OS-sandboxed mediator will make the contract adversarially tamper-proof. Similarly, while our cross-vendor experiments vary the underlying model across a shared harness, controlled component ablations toggling the typed RLM workspace, pre-registration contract, and Orchestrator steering independently are important next steps.

8 Conclusion

An autonomous research agent earns the compute it spends when four things hold: it stays coherent across long horizons, its forecasts about its own runs hold up against reality, its trace is cryptographically reproducible, and the researcher can steer it when it stalls. The Distillate harness addresses all four: across nine campaigns and $n{=}326$ pre-registered runs, modern CLI agents accurately forecast their next-run improvements ($\Delta\hat{y}{\to}\Delta y$ at $r{=}{+}0.35$, $p{<}0.001$, four times above static priors), compress transformers to 400 parameters via non-canonical weight sharing, and autonomously discover a discrete flow matching model that advances the state-of-the-art on de novo glycan structure prediction by $+11.6$ percentage points. We asked whether modern CLI agents are well-calibrated autonomous ML researchers; **the answer is yes, given the right harness**. Will agents automate research? Our hope is that, instead, agent systems like Distillate will augment it, giving scientists command over larger scopes, harder questions, and more ambitious programs. If, tomorrow, researchers see further, perhaps it will be by standing on the shoulders of agents [Newton, 1675].

References

Ejas Althaf Abtheen, Arun Singh, Shyam Sriram, Changyou Chen, Sriram Neelamegham, and Rudiyanto Gunawan. Transformer-based deep learning for glycan structure inference from tandem mass spectrometry. *bioRxiv*, 2025. doi: 10.1101/2025.07.02.662857.

Anthropic. Model Context Protocol (MCP) specification. https://modelcontextprotocol.io, 2024.

P. Ball. *Alchemy: An Illustrated History of Elixirs, Experiments, and the Birth of Modern Science*. Yale University Press, 2025. ISBN 9780300280876.

J. Beel, M.-Y. Kan, and M. Baumgart. Evaluating sakana's AI Scientist: Bold claims, mixed results, and a promising future?, 2025.

D. A. Boiko, R. MacKnight, B. Kline, and G. Gomes. Autonomous chemical research with large language models. *Nature*, 624:570–578, 2023.

R. A. Bradley and M. E. Terry. Rank analysis of incomplete block designs: I. The method of paired comparisons. *Biometrika*, 39(3/4):324–345, 1952.

A. M. Bran, S. Cox, O. Schilter, C. Baldassari, A. D. White, and P. Schwaller. Augmenting large language models with chemistry tools. *Nature Machine Intelligence*, 6(5):525–535, 2024. doi: 10.1038/s42256-024-00832-8.

J. S. Chan, N. Chowdhury, O. Jaffe, et al. MLE-bench: Evaluating machine learning agents on machine learning engineering, 2024. OpenAI.

W.-L. Chiang, L. Zheng, Y. Sheng, A. N. Angelopoulos, et al. Chatbot Arena: An open platform for evaluating LLMs by human preference, 2024.

Cognition AI. SWE-bench technical report (Devin). https://cognition.com/blog/swe-bench-technical-report, 2024.

X. Deng, J. Da, E. Pan, Y. Y. He, C. Ide, et al. SWE-Bench Pro: Can AI agents solve long-horizon software engineering tasks?, 2025. Scale AI.

Edison Scientific. Kosmos: An AI scientist for autonomous discovery, November 2025.

A. Fourney, G. Bansal, H. Mozannar, C. Tan, E. Salinas, E. Zhu, et al. Magentic-One: A generalist multi-agent system for solving complex tasks, 2024. Microsoft Research.

S. Gao, A. Fang, Y. Huang, V. Giunchiglia, A. Noori, J. R. Schwarz, Y. Ektefaie, J. Kondic, and M. Zitnik. Empowering biomedical discovery with AI agents. *Cell*, 187(22):6125–6151, 2024.

S. Grundl. A comparison of agentic AI systems and human economists. https://claude-code-economist.com/data/paper.pdf, April 2026. Working paper.

J. M. Hofman, A. Chatzimparmpas, A. Sharma, D. J. Watts, and J. Hullman. Pre-registration for predictive modeling, 2023.

K. Huang et al. POPPER: Automated hypothesis validation with agentic sequential falsifications, 2025.

N. Huntington-Klein, C. C. Pörtner, I. McCarthy, and the Many Economists Collaborative on Researcher Variation. The sources of researcher variation in economics. Technical Report Working Paper No. 33729, NBER, 2025.

Z. Jiang et al. AIDE: AI-driven exploration in the space of code, 2025.

C. E. Jimenez, J. Yang, A. Wettig, S. Yao, K. Pei, O. Press, and K. Narasimhan. SWE-bench: Can language models resolve real-world GitHub issues? In *ICLR*, 2024.

S. Kadavath, T. Conerly, A. Askell, T. Henighan, et al. Language models (mostly) know what they know, 2022.

S. Kapoor and A. Narayanan. Leakage and the reproducibility crisis in ML-based science. *Patterns*, 4(9):100804, 2023. doi: 10.1016/j.patter.2023.100804.

A. Karpathy. Open educational projects: `nanoGPT`, `makemore`, `micrograd`, and the `LLM101n` curriculum. https://github.com/karpathy, 2022–2025.

O. Khattab, A. Singhvi, P. Maheshwari, et al. DSPy: Compiling declarative language model calls into state-of-the-art pipelines. In *ICLR*, 2024.

A. Krizhevsky. Learning multiple layers of features from tiny images. Technical report, University of Toronto, 2009.

R. Lacombe. GlycoGen: Glycan structure prediction with crystallizing flows, 2026. Preprint.

R. Lacombe, K. Wu, E. Dilworth, et al. ClimateX: Do LLMs accurately assess human expert confidence in climate statements? In *NeurIPS Workshop on Tackling Climate Change with Machine Learning*, 2023.

R. Lacombe, K. Wu, E. Dilworth, et al. Don't think twice! Over-reasoning impairs confidence calibration. In *ICML Workshop on Reliable and Responsible Foundation Models*, 2025.

D. Lakens. Improving your statistical inferences—chapter 13: Preregistration and transparency. Open educational textbook, https://lakens.github.io/statistical_inferences/, 2022.

N. Lee, K. Sreenivasan, J. D. Lee, K. Lee, and D. Papailiopoulos. Teaching arithmetic to small transformers, 2023.

R. Li, D. Patel, Q. Wang, and X. Du. MLR-Copilot: Autonomous machine learning research based on large language model agents, August 2024.

S. Lin, J. Hilton, and O. Evans. Teaching models to express their uncertainty in words. *Transactions on Machine Learning Research*, 2022.

X. Liu, H. Yu, H. Zhang, Y. Xu, X. Lei, H. Lai, et al. AgentBench: Evaluating LLMs as agents. In *ICLR*, 2024.

Chris Lu, Cong Lu, R. T. Lange, J. Foerster, J. Clune, and D. Ha. The AI Scientist: Towards fully automated open-ended scientific discovery, 2024. Sakana AI.

A. Madaan, N. Tandon, P. Gupta, S. Hallinan, L. Gao, S. Wiegreffe, et al. Self-Refine: Iterative refinement with self-feedback. In *NeurIPS*, 2023.

G. Mialon, C. Fourrier, C. Swift, T. Wolf, Y. LeCun, and T. Scialom. GAIA: A benchmark for general AI assistants, 2023.

I. Newton. Letter to Robert Hooke, 1675. Dated 5 February 1675; published in H. W. Turnbull (ed.), *The Correspondence of Isaac Newton, Vol. 1*, Cambridge University Press, 1959, pp. 416–417.

A. Novikov, N. V u, M. Eisenberger, et al. AlphaEvolve: A coding agent for scientific and algorithmic discovery, 2025. Google DeepMind.

C. Packer, S. Wooders, K. Lin, V. Fang, S. G. Patil, I. Stoica, and J. E. Gonzalez. MemGPT: Towards LLMs as operating systems, 2023.

D. Papailiopoulos and contributors. AdderBoard: A community leaderboard for the smallest transformer that can add two ten-digit numbers. https://github.com/anadim/AdderBoard, 2024.

J. S. Park, J. C. O'Brien, C. J. Cai, M. R. Morris, P. Liang, and M. S. Bernstein. Generative agents: Interactive simulacra of human behavior. In *UIST*, 2023.

S. G. Patil, T. Zhang, X. Wang, and J. E. Gonzalez. Gorilla: Large language model connected with massive APIs, 2023.

B. Romera-Paredes, M. Barekatain, A. Novikov, M. Balog, M. P. Kumar, E. Dupont, F. J. R. Ruiz, J. S. Ellenberg, P. Wang, O. Fawzi, P. Kohli, and A. Fawzi. Mathematical discoveries from program search with large language models. *Nature*, 625:468–475, 2024.

T. Schick, J. Dwivedi-Yu, R. Dessì, R. Raileanu, M. Lomeli, E. Hambro, L. Zettlemoyer, N. Cancedda, and T. Scialom. Toolformer: Language models can teach themselves to use tools. In *NeurIPS*, 2023.

S. Schmidgall, Y. Su, Z. Wang, X. Sun, J. Wu, X. Yu, J. Liu, M. Moor, Z. Liu, and E. Barsoum. Agent Laboratory: Using LLM agents as research assistants, 2025.

H. Semmelrock et al. Reproducibility in machine-learning-based research: Overview, barriers, and drivers. *AI Magazine*, 46(2):e70002, 2025.

Y. Shen, K. Song, X. Tan, D. Li, W. Lu, and Y. Zhuang. HuggingGPT: Solving AI tasks with ChatGPT and its friends in Hugging Face. In *NeurIPS*, 2023.

N. Shinn, F. Cassano, E. Berman, A. Gopinath, K. Narasimhan, and S. Yao. Reflexion: Language agents with verbal reinforcement learning. In *NeurIPS*, 2023.

K. Swanson, W. Wu, N. L. Bulaong, J. E. Pak, and J. Zou. The Virtual Lab: AI agents design new SARS-CoV-2 nanobodies with experimental validation. *bioRxiv*, 2024. doi: 10.1101/2024.11.11.623004.

T. H. Trinh, Y. Wu, Q. V. Le, H. He, and T. Luong. Solving olympiad geometry without human demonstrations. *Nature*, 625:476–482, 2024. doi: 10.1038/s41586-023-06747-5.

G. Wang, Y. Xie, Y. Jiang, et al. Voyager: An open-ended embodied agent with large language models, 2023.

H. Wijk, T. Lin, J. Becker, S. Jawhar, N. Parikh, et al. RE-Bench: Evaluating frontier AI R&D capabilities of language model agents against human experts, 2024. METR.

Q. Wu, G. Bansal, J. Zhang, Y. Wu, B. Li, E. Zhu, L. Jiang, X. Zhang, S. Zhang, J. Liu, A. H. Awadallah, R. W. White, D. Burger, and C. Wang. AutoGen: Enabling next-gen LLM applications via multi-agent conversation, 2023.

Y. Yamada, R. T. Lange, et al. The AI Scientist-v2: Workshop-level automated scientific discovery via agentic tree search, 2025.

S. Yao, D. Yu, J. Zhao, I. Shafran, T. L. Griffiths, Y. Cao, and K. Narasimhan. Tree of Thoughts: Deliberate problem solving with large language models. In *NeurIPS*, 2023a.

S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao. ReAct: Synergizing reasoning and acting in language models. In *ICLR*, 2023b.

A. L. Zhang, T. Kraska, and O. Khattab. Recursive Language Models, 2025.

A. L. Zhang, Z. Li, and O. Khattab. The mismanaged geniuses hypothesis. Blog post, https://alexzhang13.github.io/blog/2026/mgh/, April 2026.

S. Zhou, F. F. Xu, H. Zhu, X. Zhou, R. Lo, A. Sridhar, et al. WebArena: A realistic web environment for building autonomous agents. In *ICLR*, 2024.

Responsible use

Autonomous ML research systems compress the cost of running experiments, which cuts both ways: harnesses like Distillate lower the barrier for rigorous pre-registered campaigns, but the same infrastructure could automate dual-use research or generate misleading artifacts at scale, and multi-day agent campaigns consume non-trivial energy that should be scoped and reported. Distillate embeds two mitigations we recommend as defaults: cryptographic pre-registration makes agent reasoning auditable by any third-party reader walking the Git history; bidirectional human-in-the-loop steering gives researchers a low-friction path to halt or redirect an experiment as it unfolds. Sandboxed enforcement, formal governance of what agents may research autonomously, and honest per-campaign compute reporting belong to the broader community response.

A System prompt excerpts

Both agents' full system prompts ship with the released harness and are unit-test-snapshotted; excerpts below capture the load-bearing sections. Placeholders in `{{double-braces}}` are interpolated at runtime.

Orchestrator (Nicolas).

> *You are **Nicolas**, a research alchemist — named after Nicolas Flamel. Do NOT identify as Claude Code or Claude. Your name is Nicolas. You are the command and control center for a researcher's experimental work. . . .*
>
> **Lab REPL — Your Primary Analysis Tool.** You have a `mcp__distillate__lab_repl` tool: a persistent Python sandbox for multi-step reasoning. USE IT whenever a question spans multiple papers AND experiments, requires filtering / ranking / comparison, needs multi-step analysis, or would otherwise require 3+ sequential tool calls. Available: `lab.papers.{search, get, recent, queue, stats, highlights}`, `lab.experiments.{list, get, runs, active}`, `lab.notebook.{recent, digest}`, `llm_query(...)` (fast Haiku sub-call), `delegate(...)` (recursive sub-agent with data), `FINAL(answer)`. Always provide a human-readable `description`. For quick single lookups, use specific MCP tools directly.
>
> *Reflex: search before admitting ignorance. Before saying "I don't know" about any named model / paper / dataset / method, first call `lab.papers.search("<name>")` and `lab.experiments.active()`; only ask the user to fill in specs after the library search comes up empty.*

Experimentalist.

> *You are an **Experimentalist** — an autonomous research agent of the Distillate Lab, instantiated to advance a specific research goal. Read `PROMPT.md` and follow it precisely.*
>
> You are fully autonomous. The human may be asleep. Do not pause, do not ask questions, do not wait for input. Work indefinitely until manually stopped. If you are stuck, try a different approach.
>
> **Pause protocol.** If `start_run` returns `{"paused": true}`, the user has paused this experiment to ponder steering. Do NOT call `start_run` again — exit the session cleanly. The user will remove `.distillate/pause_requested` (often after writing fresh `.distillate/steering.md`) and resume you in a new session.
>
> **Run Protocol.** For EVERY run: *Step 0 Plan.* Read `runs.jsonl` and `context.md`; build on what worked, avoid repeating failures. Pre-register by calling `start_run(...)` (schema in Table 1). The prediction must be concrete and falsifiable. *Step 1 Train.* Launch training through `distillate-run` (reads `.distillate/budget.json`, sends SIGTERM at the budget with a grace window). Never hardcode `MAX_SECONDS`; read it via `distillate.budget.read_train_budget()`. Print metrics incrementally. *Step 2 Conclude.* Call `conclude_run(run_id, actual_value, verdict, belief_update)`; the tool emits the merge commit and pushes.

B The .distillate filesystem layout

Each Distillate experiment is its own Git repository. A reserved `.distillate/` directory at the root holds all agent state:

```
my-experiment/
|-- PROMPT.md                # Orchestrator's brief (Question /
|                            #   Context / Optimization / Budget)
|-- train.py                 # training script
|-- .distillate/
|   |-- runs.jsonl           # append-only, prereg + conclude rows
|   |-- events.jsonl         # per-epoch metric stream from training
|   |-- context.md           # rolling Reflexion synthesis
|   |-- scratchpad.md        # experimentalist working memory
|   |-- budget.json          # train + wrap deadlines (per-run)
|   |-- steering.md          # researcher / Orchestrator hints
|   |-- pause_requested      # sentinel; refuses next start_run
|   |-- experiment.json      # metric_name, draft flag, overrides
|   |-- calibration.jsonl    # (confidence, verdict) probe pairs
|   |-- cost.json            # sub-LM ledger {model: total_usd}
|   `-- checkpoints/<run_id>/ # artefact cache (SHA-256 in conclude)
`-- .git/                    # code AND .distillate/ are tracked
```

Two commits per run. Each run produces two cryptographically linked Git commits. `start_run` fast-forward-rebases onto `origin/HEAD`, commits a `prereg:` entry staging the new `runs.jsonl` row and `budget.json`, and *pushes to origin before training launches*, anchoring the prediction in GitHub's receive-time and the content-addressed Git SHA. `conclude_run` merges `origin/HEAD` into the index without committing, then emits a `run:` merge commit whose parents are the prereg and the orchestrator's contemporary state. Trailers (`Distillate-Kind`, `Distillate-Run`, `Distillate-Verdict`, `Distillate-Metric`) make the trace machine-filterable. Two distinct authors (`Nicolas <nicolas@distillate.dev>` and `Experimentalist <experimentalist@distillate.dev>`) separate orchestration from execution in `git shortlog`.